EDGE AI and FlagOS

Models, devices and complete systems

13 October 2026

The two-hour course connects six decisions

Each section uses the answer from the previous one.

FIRST HOUR

01 Workloads and constraints
What runs locally, and what limits it?

02 Architecture choices
How do CPU and accelerator choices differ?

03 Runtimes and local systems
How does a model use the chosen hardware?

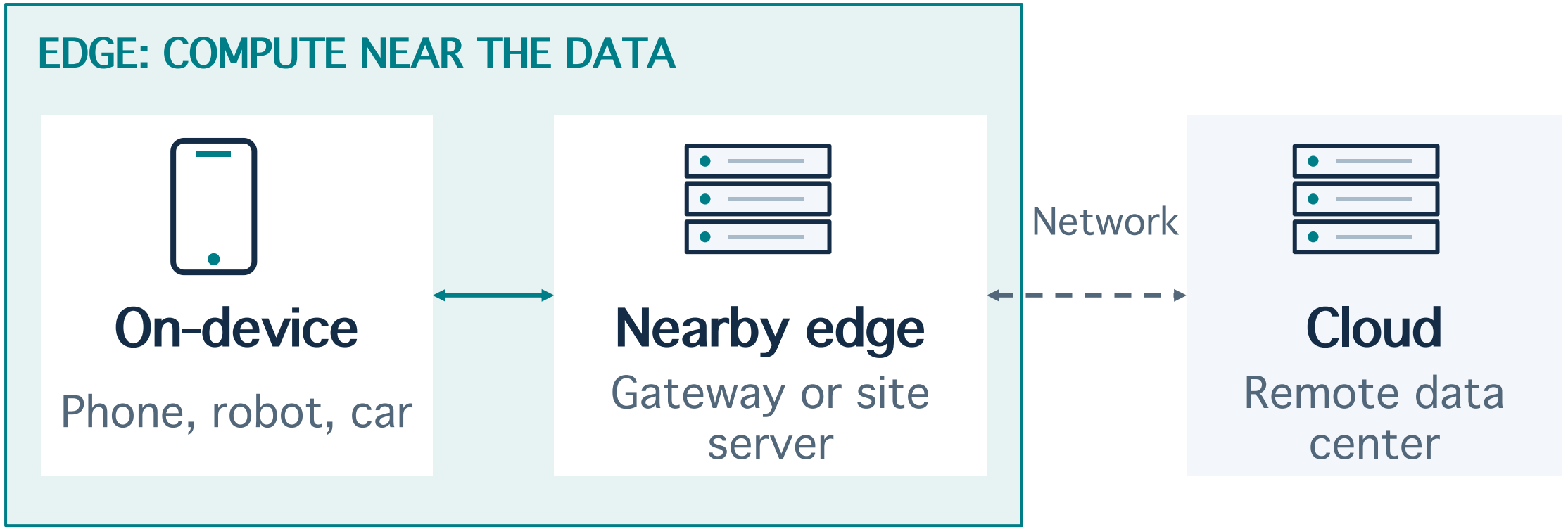
SECOND HOUR

04 FlagOS and partner platforms
What can FlagOS offer for Edge AI?

05 Discussion and takeaways

Two discussion periods are included in the 120-minute schedule

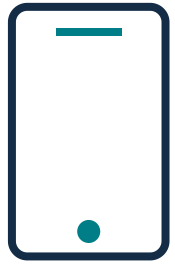
On-device AI is one part of the edge



Work can stay local or use a hybrid path

Conceptual placement map. Sources: NIST SP 500-325 and Google AI Edge, linked in notes.

Edge AI takes many physical forms



Personal device

Phone or wearable

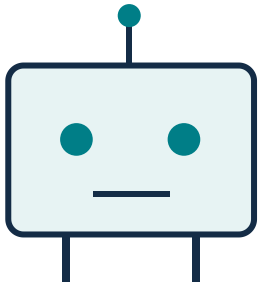
Speech assistance



Industrial edge

Camera or gateway

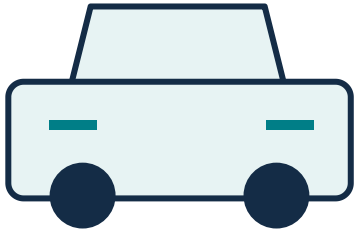
Visual inspection



Robot

Mobile platform

Navigation support



Vehicle

Cockpit AI box

Local assistance

Illustrative application map. Deployment choices depend on the device and workload.

Example: object-level scene analysis

Extreme Ironing: Panoptic Segmentation Using CNNs

EMBEDDED VISION SUMMIT

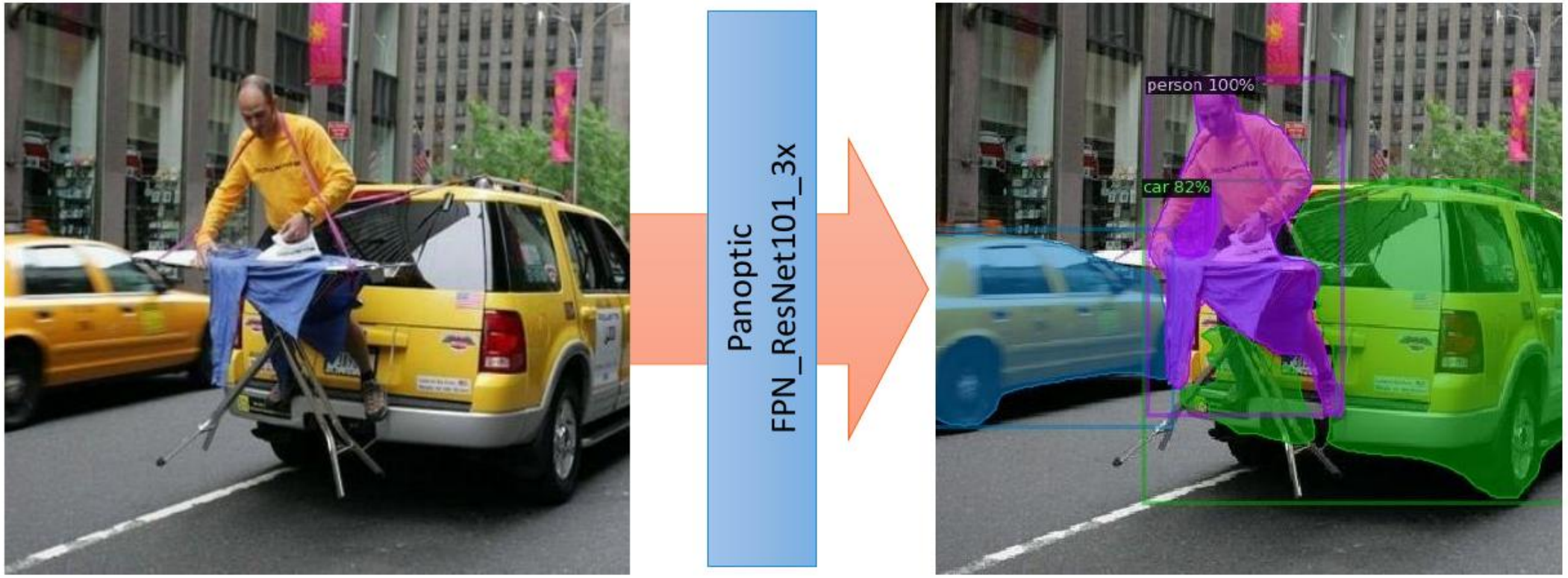


Image source: https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg
Model Used: Detectron2 - COCO-PanopticSegmentation/panoptic_fpn_R_101_3x

Example: a vision-language explanation

Extreme Ironing: Multimodal Transformers Provide Better Contextual Awareness

EMBEDDED VISION SUMMIT



LLaVA (Large Languag & Vision Assistant)

User: What is unusual about this image?

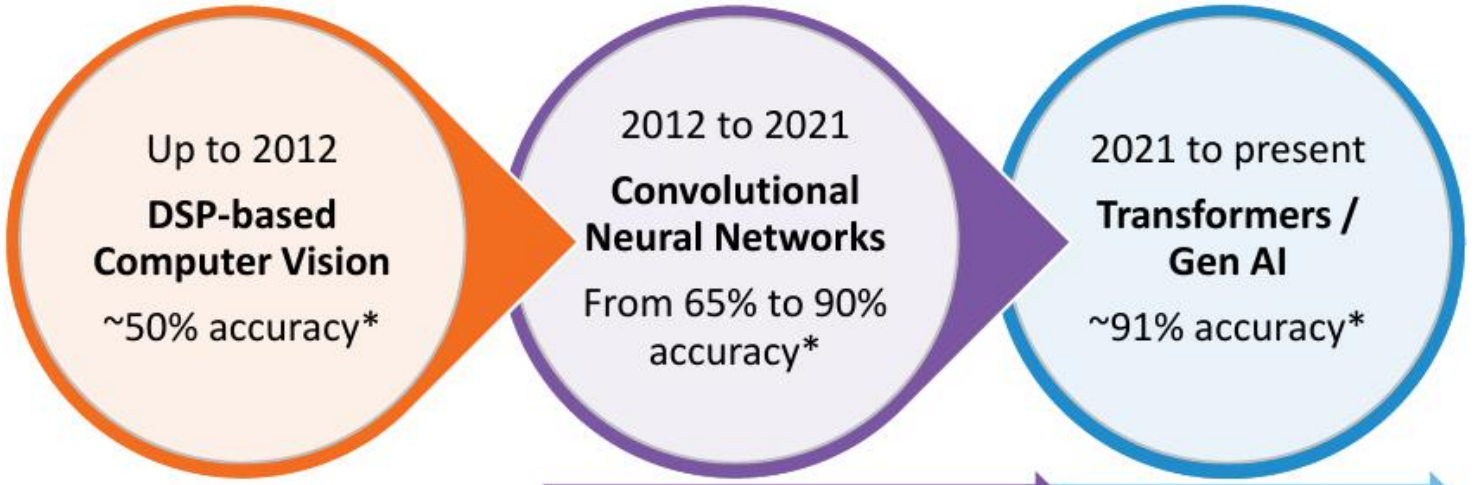
LLaVA: The unusual aspect of the image is that a man is ironing clothes on the back of a yellow minivan while it is on the road. This is an unconventional and unsafe place to perform such an activity, as ironing clothes typically requires a stable surface and appropriate equipment. Ironing clothes in a moving vehicle could lead to potential hazards for both the person doing the ironing and other road users.

Image source: https://arxiv.org/pdf/2304.08485.pdf

4

Historical vendor view: changing model workloads

Challenge: AI/ML Technology Evolving (Moving Target)



- Residual connections
- Inception
- Depthwise separable convolutions
- Squeeze and Excitation layers
- New activation functions

*ImageNet Top-1 Accuracy

© 2025 Synopsys Inc.

5

MoE
(Mixture-of-Experts)

Uses a collection of smaller expert networks, each specialized in different aspects of the input, to improve performance and efficiency

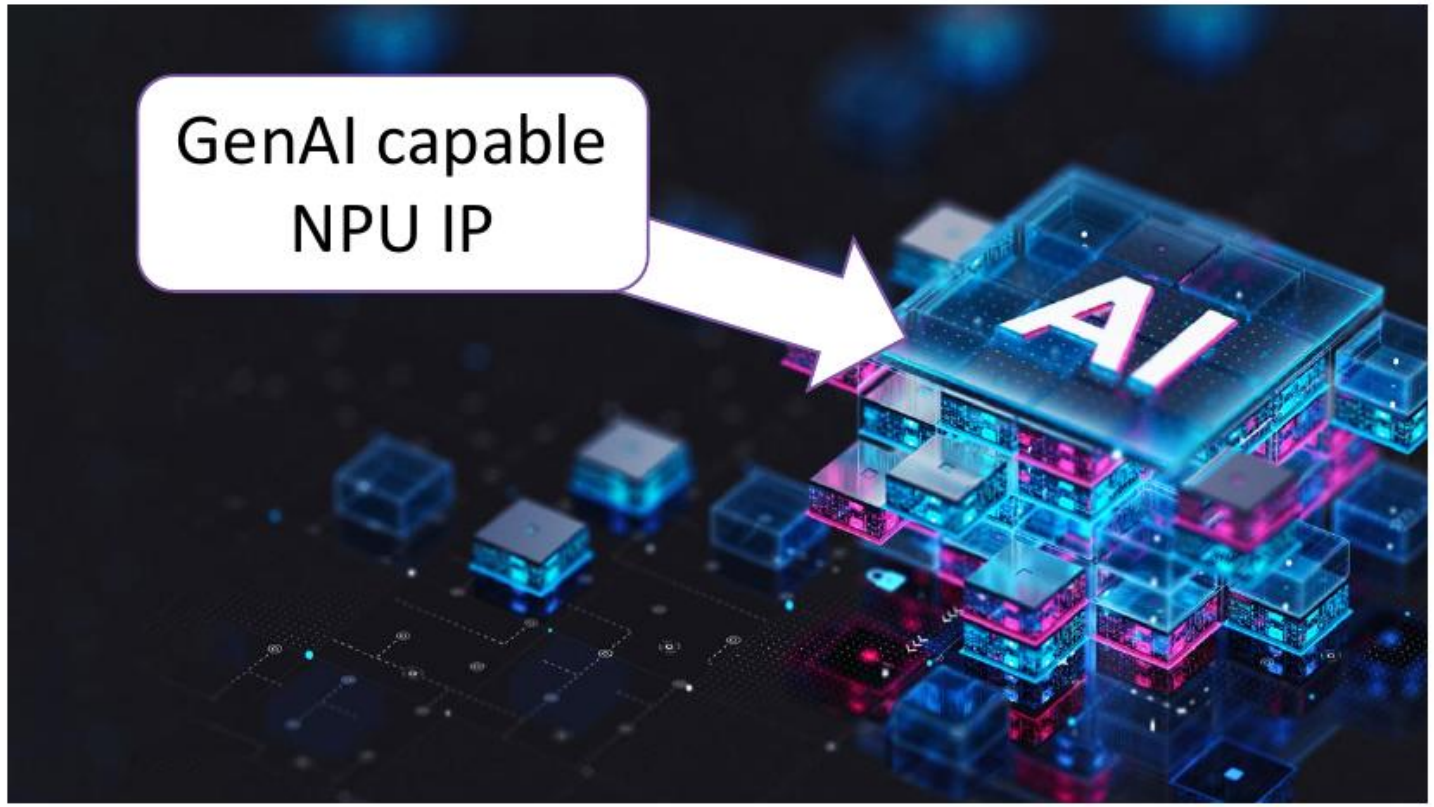
Concept originated from the 1991 paper *Adaptive Mixture of Local Experts*.

Used in Deepseek, Llama-4, etc.

Synopsys design premise: a GenAI-capable SoC

The Challenge of Fitting GenAI into an Edge Device SoC

EMBEDDED VISION SUMMIT



Assumptions

- Target solution is an AI-enabled SoC
- GenAI (built on transformer models) capabilities needed
- NPU is needed for transformers / GenAI performance/power efficiency

2025 high-end forecast, rather than a universal device requirement

Challenge: AI / ML Requirements for AI SoCs Rising

EMBEDDED VISION SUMMIT

	Last 5 years	Ongoing Designs	Next 3 years
Algorithms	CNNs, RNNs	Transformers, GenAI (Image Gen, LLMs)	Transformers, GenAI (LVMs, LMMs, SLMs)
High End M/L Performance on the edge	100s of TOPS	Up to 1000 TOPS	2000+ TOPS
NPU Data Types	INT8	INT8 / INT4 FP16 / BF16	INT4 / INT8 FP4, FP8, OCP MX
Multi-Die/Chiplet	N / A	UCIe v1.1	UCIe v1.2
Typical Process Nodes*	16 nm / 12 nm	7 nm / 5 nm / 3 nm	3 nm / 2 nm

**ARC Processor IP (NPX6) is process node agnostic*

Every edge device has four budgets



Memory

7B × 4 bits ÷ 8 ≈ 3.5 GB

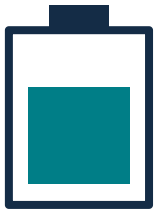
Raw weights only

Add KV cache, scales and workspace

Memory → Compute

Bandwidth

Moving data can limit speed



Power and heat

Sustained load matters



Latency

The whole response path counts

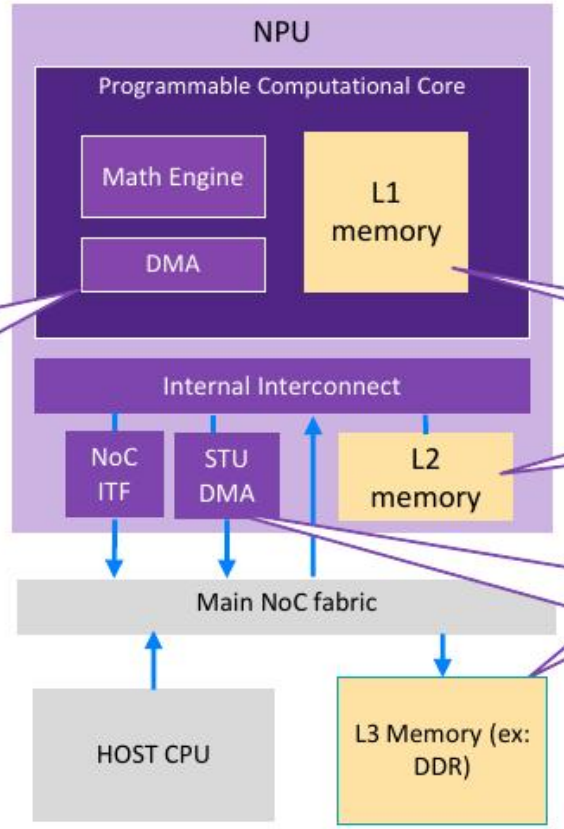
7B calculation uses decimal GB and ideal 4-bit packing. Sources and assumptions in notes.

Architecture example: compute and memory work together

Key Architecture Considerations for NPUs Running GenAI

EMBEDDED VISION SUMMIT

- ✓ Requires software tools that support HW features and rapid architecture exploration
- ✓ Requires a Programmable Solution Designed for latest Transformers



- ✓ Requires Bandwidth reduction in hardware and software
- ✓ Requires multi-level memory management
- ✓ Requires Low-bit resolution support (INT4, FP4, FP6 etc.) for data transfers to minimize BW

Architecture choices under edge constraints

The Synopsys examples identify chip demands. The next decision is where each operation executes.

CPU instruction-set choices

Arm and RISC-V

Instructions visible to CPU software

Vector and matrix capabilities shape CPU kernels

Complete SoC execution system

CPU, GPU and NPU resources

Memory hierarchy and data movement

Compiler and operator coverage

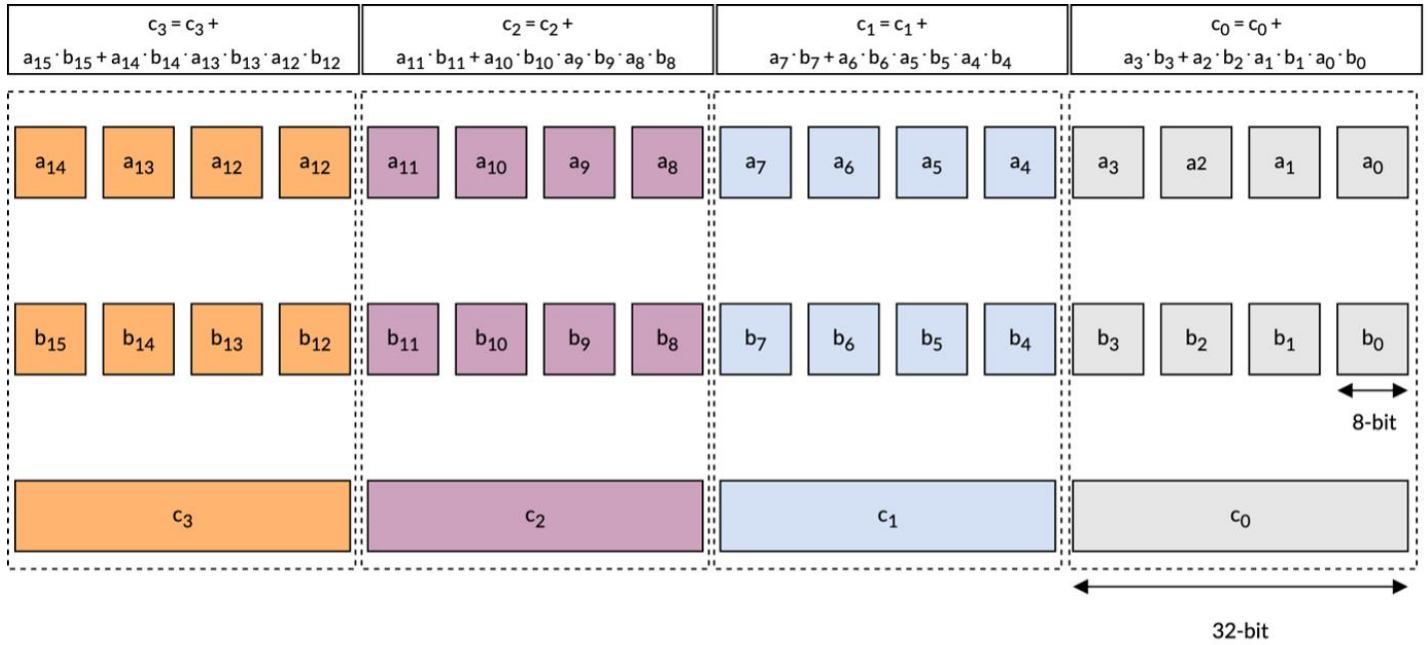
Power under sustained load

The comparison: CPU kernels, CPU–NPU systems, then extensible RISC-V AI subsystems

Arm CPU inference with KleidiAI

Optimized microkernels connect CPU instructions to familiar ML frameworks.

ExecuTorch + XNNPACK + KleidiAI



Quantized matmul

SDOT and I8MM accelerate the arithmetic used by quantized models.

Existing devices

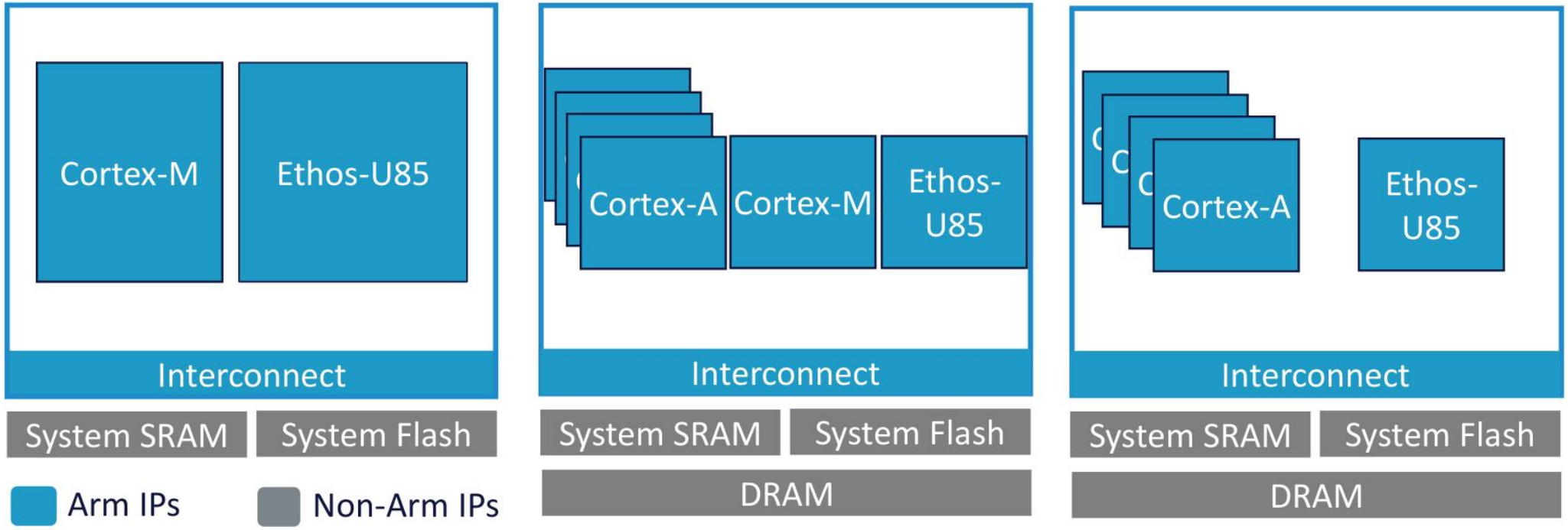
The article covers phones and Raspberry Pi 5 running local language models.

Source: Arm and Meta, PyTorch blog, 11 August 2025

Arm CPU and NPU system designs

Ethos-U85 can serve MCU endpoints or systems with Cortex-A processors.

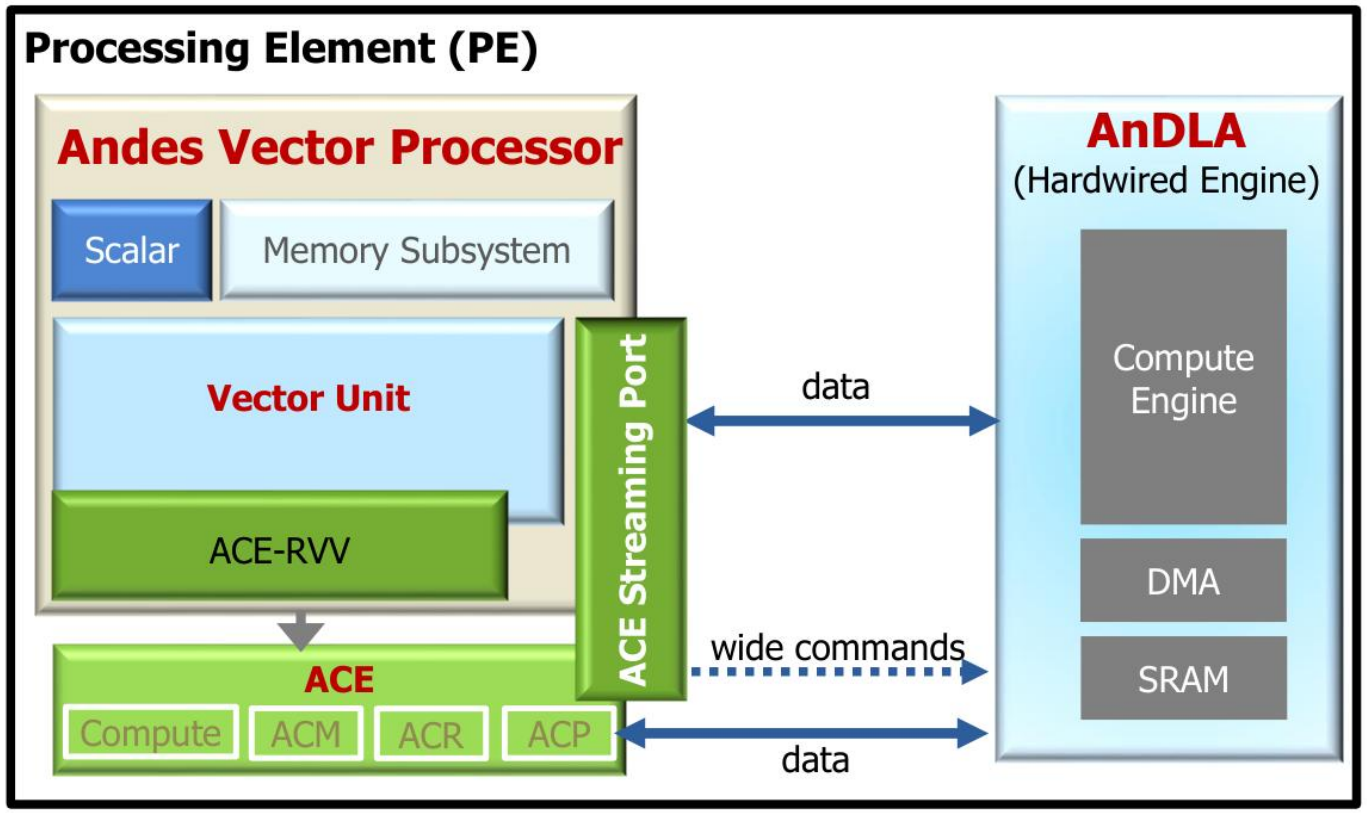
- End Point AI: Cortex-M based system
- ML Island: Cortex-A based system
- Discrete NPU: Cortex-A only



Source: Arm, Embedded Vision Summit 2024, PDF p.10

Andes: extensible RISC-V AI subsystems

A vector processor can cooperate with a dedicated deep-learning accelerator.



Compute extensions

Custom instructions add operations around the vector processor.

Data movement

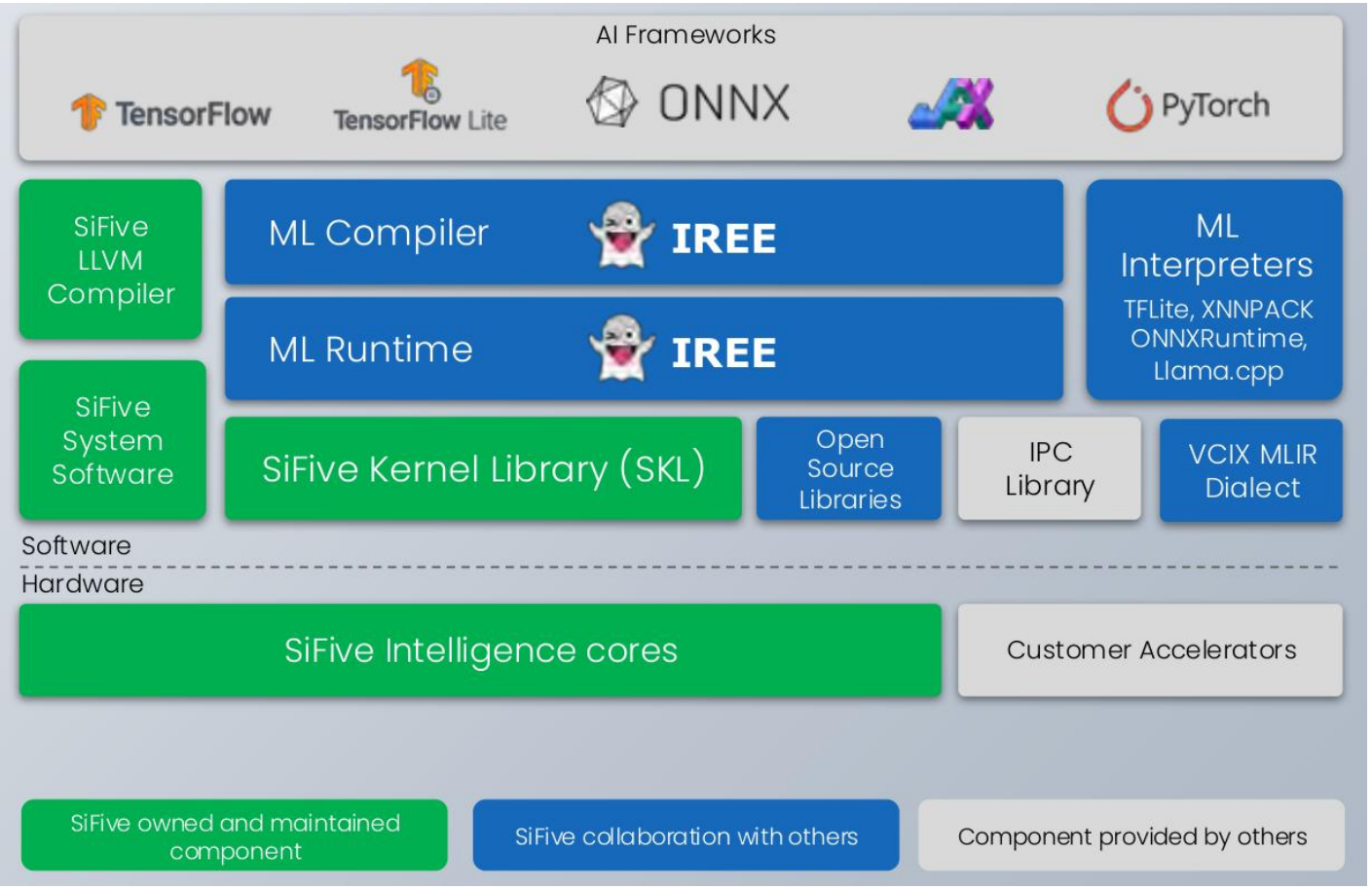
Streaming interfaces connect programmable compute with AnDLA.

AndesAIRE provides the software layer above this subsystem.

Source: Andes, AI on RISC-V (2025), PDF pp.20–21

SiFive: software above the AI hardware

Framework integration depends on compiler, runtime and kernel-library support.



Compiler path

IREE and MLIR connect models to the target compute system.

Kernel path

SKL and framework integrations provide optimized implementations.

The original figure distinguishes vendor and ecosystem responsibilities.

Source: SiFive, RISC-V Day Tokyo 2025, PDF pp.26–27

Inference engines connect models to hardware

The architecture exposes capabilities. The software path determines which ones the model can actually use.

What the platform provides

CPU instructions and optimized kernels

Accelerator compiler and runtime

Memory, copies and synchronization

What the engine must handle

Model format and supported operators

Placement, layouts and precision

Execution order and fallback behavior

Next: compare ONNX Runtime, llama.cpp and LiteRT by their integration boundary

Five engines, five integration paths

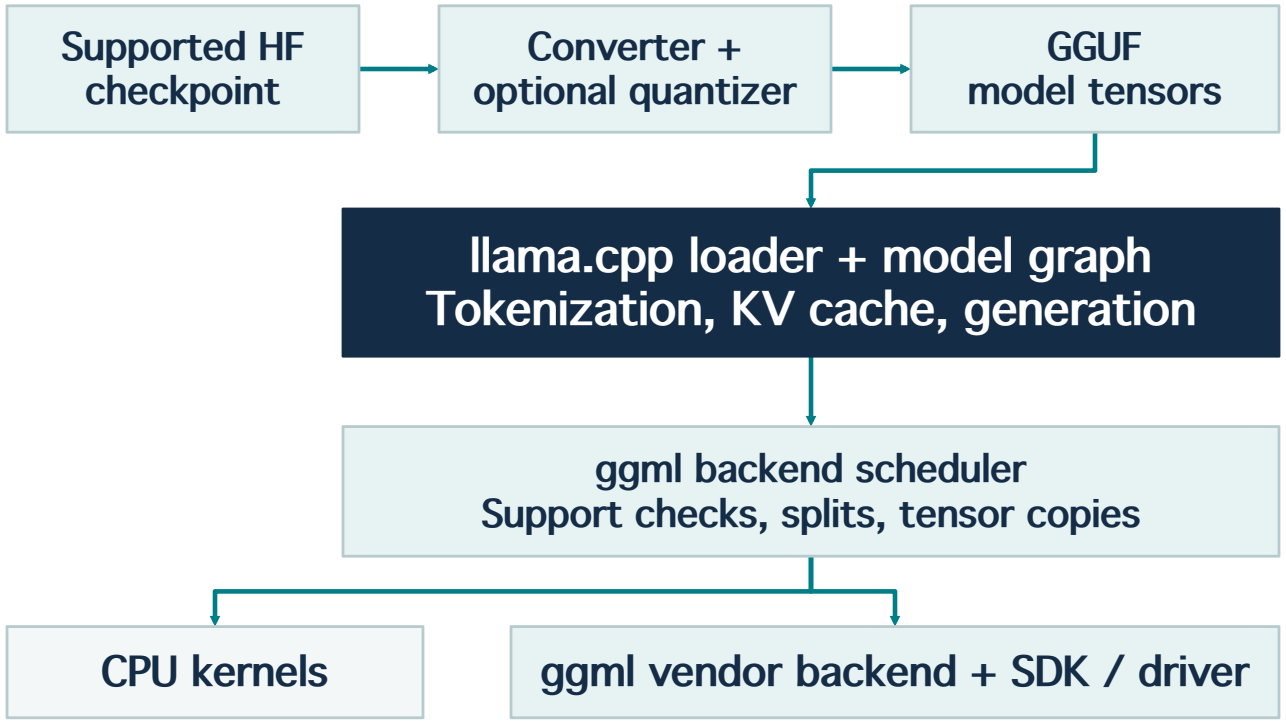
Choose by workload, deployable artifact and the hardware route you can actually support

Engine	Typical workload	Source / export → runtime artifact	Hardware route	Vendor entry point
llama.cpp	Local LLMs Supported VLMs	Supported checkpoint → GGUF Architecture-specific conversion	CPU + ggml backends Hybrid execution	ggml backend Kernels + buffer API
LiteRT	Mobile / embedded Vision and audio	TF / PyTorch / JAX → .tflite Optional vendor bytecode	CPU / GPU Partner-specific NPU	Compiler + Dispatch plugins Or Interpreter delegate
ONNX Runtime	General graphs Edge / server	Framework exporter → ONNX Optional ORT format	CPU + selected EPs GPU / NPU by provider	Execution Provider (EP) Capability + compile / kernels
TensorRT	NVIDIA-targeted inference	ONNX or network API → serialized engine plan	NVIDIA GPU Supported Jetson DLA	Custom operator plugin NVIDIA execution target
ExecuTorch	PyTorch edge Audio / CV / GenAI	torch.export → Edge lowering → .pte (+ optional .ptd)	CPU + linked delegates GPU / NPU by backend	AOT partitioner + compiler Runtime backend interface

llama.cpp

Local LLM inference with model-specific graphs, low-bit weights and CPU / accelerator splitting

MODEL CONVERSION



Vendor integration contract

- **Operators and kernels**

  Implement supports_op and graph_compute.
  Match GGUF quantization layouts and model ops.

- **Buffers and synchronization**

  Expose buffer types, allocation and tensor copies.
  Implement async execution / events where used.

- **Partitioning and loading**

  Scheduler splits supported work across backends.
  CPU coverage enables fallback, with transfer cost.

Data: FP16 / BF16 and GGUF Q4 / Q8 blocks
Exact layout and kernel support vary by backend.

Hardware conditions

CPU: Arm / x86. GPU: CUDA, Metal, Vulkan, SYCL.
NPU support requires a specific backend and SDK.

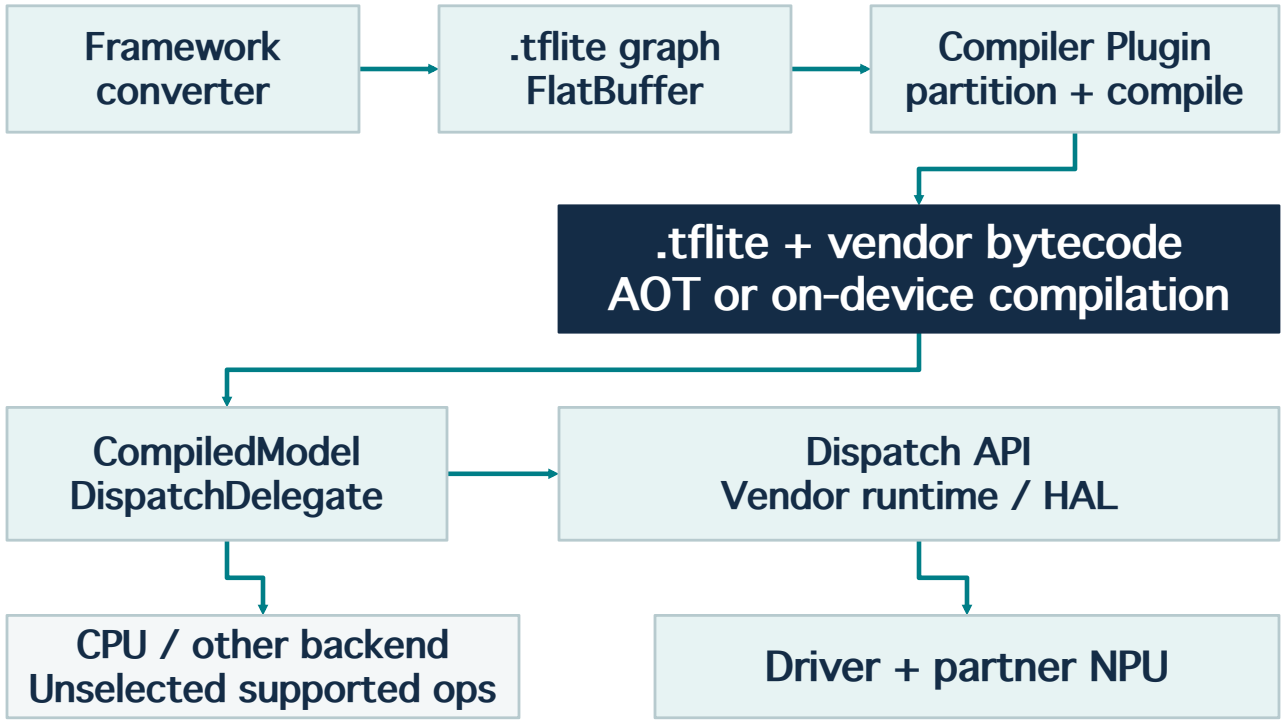
Toolchain openness

Provide a ggml adapter and compatible SDK / driver.
Binary vendor libraries can remain proprietary.

LiteRT

Compact edge runtime with graph delegation and vendor compilation for supported accelerators

TF / PYTORCH / JAX CONVERSION + NPU COMPILATION



Vendor integration contract

- **Compiler and operator coverage**
  Compiler Plugin selects subgraphs and emits bytecode.
  Declare shapes, dtypes and supported operators.
- **Runtime and memory**
  Dispatch API loads code and binds device buffers.
  Expose invocation, buffer import and completion.
- **Alternative integration route**
  Interpreter delegates replace supported graph regions.
  Remaining ops need available CPU / other kernels.

Data: FP32 / FP16 / INT8 tensors
Quantization and shape support depend on the backend.

Hardware conditions

CPU and GPU paths depend on platform and build.
NPU acceleration requires an integrated vendor stack.

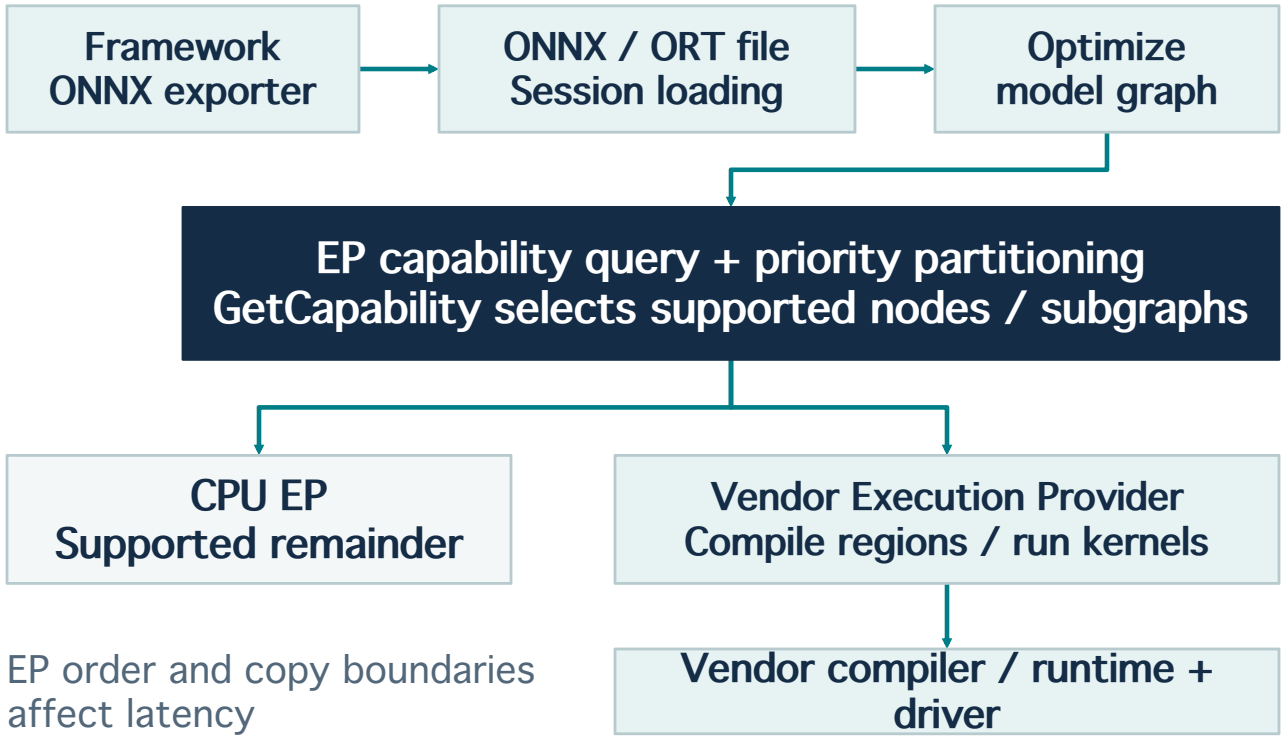
Toolchain openness

Ship compiler + dispatch libraries, headers and SDK.
Vendor compiler / HAL source need not be public.

ONNX Runtime

General graph execution with provider-based partitioning across CPU, GPU and vendor accelerators

EXPORT AND SESSION INITIALIZATION



EP order and copy boundaries affect latency

Vendor integration contract

- **Capability and execution**

  Implement EP kernels or compile fused subgraphs.
  Declare supported opsets, shapes and dtypes.

- **Memory and synchronization**

  Provide allocators, device buffers and data transfers.
  Integrate stream sync and supported I/O binding.

- **Artifacts and fallback**

  Load ONNX / ORT, plus optional EP-compiled caches.
  Unclaimed nodes go to a lower-priority EP, often CPU.

Data: ONNX tensor types, QDQ / QOperator quantization
Low-bit support depends on operators and EP kernels.

Hardware conditions

CPU is built in. GPU / NPU support comes from EPs such as CUDA, TensorRT, OpenVINO and QNN.

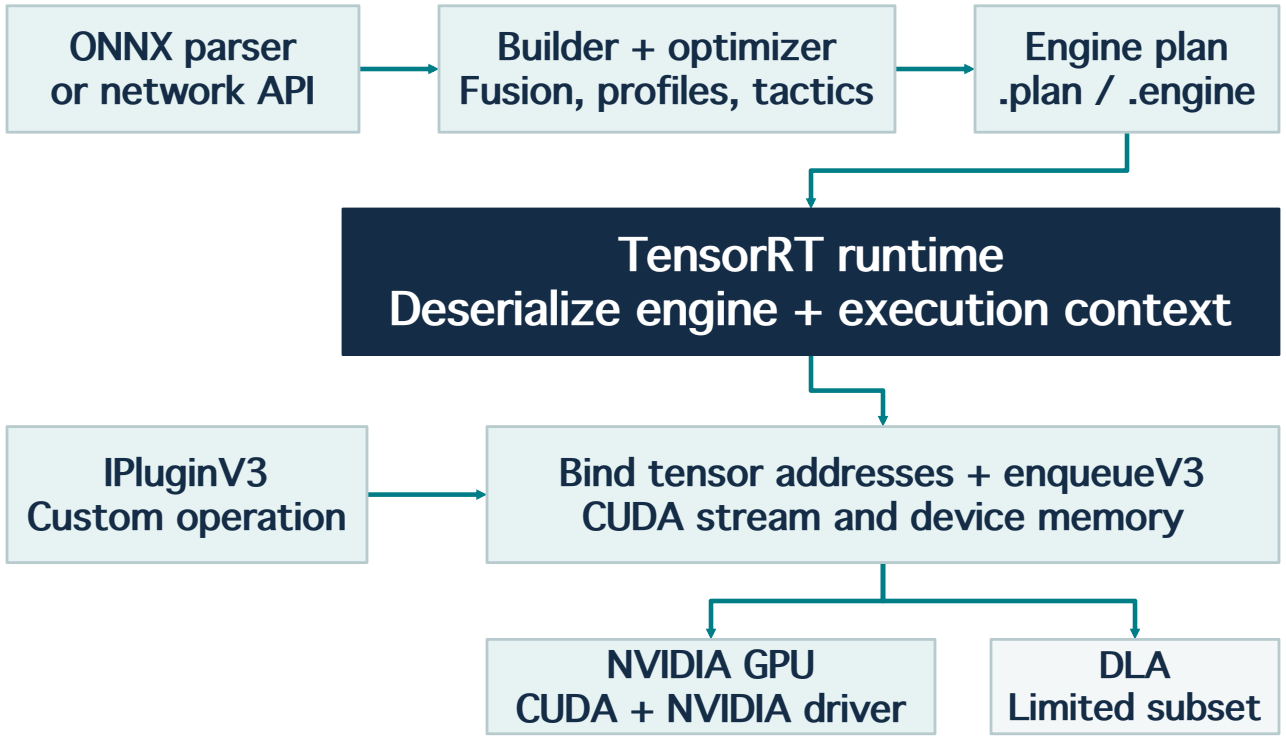
Toolchain openness

Provide an EP library with its SDK / driver dependencies.
A proprietary compiler/runtime can sit behind the EP.

TensorRT

NVIDIA inference compiler and runtime. TensorRT-LLM is a separate LLM stack.

BUILD ONCE, LOAD A COMPATIBLE ENGINE



Vendor integration contract

- **Custom operations**

  IPluginV3 defines shapes, formats and enqueue.
  Custom operators still run on NVIDIA targets.

- **Memory and execution**

  Manage device tensors, workspace and CUDA streams.
  Load plugin libraries before engine deserialization.

- **Compatibility and fallback**

  Plans require compatible runtime / hardware settings.
  DLA can use GPU fallback. No general CPU fallback.

Data: FP32 / FP16 / BF16 + explicit INT8 / FP8 / FP4
INT4 weights and low-bit paths depend on GPU / ops.

Hardware conditions

NVIDIA GPUs and supported Jetson DLA only.
No generic third-party hardware-backend interface.

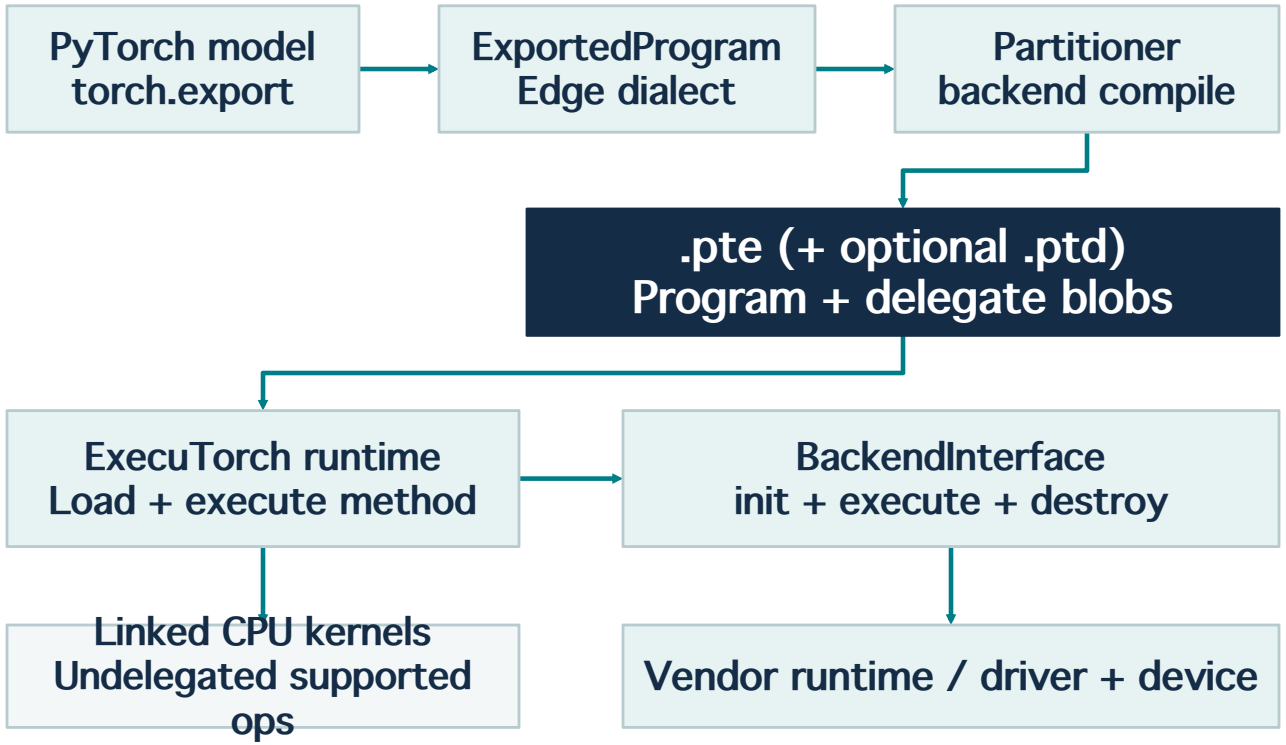
Toolchain openness

NVIDIA SDK / CUDA driver binaries are required.
OSS parser/plugins accompany a binary NVIDIA core.

ExecuTorch

PyTorch edge deployment with ahead-of-time lowering, delegated subgraphs and a small runtime

AHEAD-OF-TIME EXPORT AND LOWERING



Vendor integration contract

- **Export and compilation**

  Partition Edge operators and preprocess each region.
  Embed compiled backend data in the program.

- **Buffers and synchronization**

  Honor planned tensors and backend-owned memory.
  Define buffer lifetime, copies and completion.

- **Runtime and fallback**

  Register / link the backend and required CPU kernels.
  AOT leaves unsupported regions outside the delegate.

  Data: FP32 / FP16, quantized INT8 / INT4 paths
  Supported layouts and quantizers depend on the backend.

Hardware conditions

Portable CPU / XNNPACK plus selected backends.
GPU / NPU routes depend on the linked vendor SDK.

Toolchain openness

Provide exporter integration and a runtime backend.
Compiler / runtime binaries can remain proprietary.

Key Takeaways

Engine selection follows the deployment path and measured behavior on the target device

- **Workload, deployable artifact and actual target define the shortlist**
  Match the model family and export path to a specific backend, SDK and device.
- **Model formats carry different runtime contracts**
  .gguf, .tflite, .onnx / .ort, .plan and .pte are not interchangeable deployment artifacts.
- **Vendors must provide a complete, usable integration**
  Backend adapter, op / dtype coverage, compiler / runtime SDK, and memory / sync APIs.
  Public adapters can call proprietary vendor libraries.
- **Partitioning, fallback and copies shape end-to-end latency**
  Inspect where every graph region runs, which CPU kernels exist and what crosses device boundaries.
- **Comparable measurements require a fixed deployment test**
  Fix model, precision and inputs. Check cold / warm latency, CPU fallback, peak memory and quality.
  Use batch 1 / 2 where relevant to the workload and supported by the chosen backend.

Recommended selection sequence

Shortlist a deployment path → Verify the vendor contract → Benchmark on target → Select

A shared software problem across edge platforms

Vehicles, robots and local assistants reuse models, while their hardware and software interfaces differ.

What repeatedly changes

Supported operators and precision

Target compiler and kernel formats

Runtime services and vendor SDKs

What FlagOS aims to reuse

Framework and engine adapters

FlagGems operator implementations

FlagTree compilation and backend work

A common interface still needs target-specific correctness, coverage and performance evidence

Why the FlagOS software stack matters

AI chips need a complete software stack to join the AI ecosystem

An open, unified stack lets model support and optimizations carry across chips, from frameworks to hardware execution.

Key layer	CUDA ecosystem	FlagOS open stack
Frameworks	PyTorch / vLLM, etc.	Unified plugins for PyTorch / vLLM, etc.
Operators / comms	cuBLAS / cuDNN NCCL	FlagGems FlagCX
Code / compiler	CUDA C CUDA compiler	Triton / Triton-TLE FlagTree
Hardware	NVIDIA GPU	Multiple vendors and architectures CPU / GPU / NPU

01 Less duplicate integration

Unified framework plugins and chip backends reduce N×M integrations to N+M.

02 More chip performance

Shared operators and compiler optimizations tuned for each architecture.

03 Rapid model support

KernelGen generates kernels.
FlagRelease ports and releases models.

One open stack for diverse AI chips

FlagOS: SW Stack for Open Computing - Run AI Everywhere

The AI System Software Stack Supporting the Largest Number of Chip Types Worldwide

FlagOS is a unified, open-source AI system software stack that enables AI models and applications to run seamlessly across diverse hardware platforms.

20+ Chip Vendors

30+ AI Chips

Various AI models

Language models | Multimodal models | World models | Embodied AI

Unified Device Plugin for Training/Inference frameworks

Megatron-FL | Transformer Engine-FL | vLLM-plugin-FL | VeRL-plugin-FL | SGLang-plugin-FL | Pytorch-plugin-FL

Domain solution

FlagOS-Robo | FlagQuantums

FlagOS Skills

Deploy, code-gen, migration, benchmark

FlagOS core libraries

FlagGEMs operator library

FlagGEMs - vLLM fused-kernels

FlagGEMs, FlagBLAS, FlagSparse, FlagFFT, FlagTensor, FlagAudio

FlagScale parallel framework

FlagTree : Unified AI compiler

FlagTree: AI chips | FlagTree-CPU：ARM

FlagCX Unified comm. lib

FlagOS tools

KernelGen: operator auto-gen tool

FlagRelease: Model auto-migration tool

FlagPerf: AI chip evaluation tool

AI Compute

Servers of different AI chipsets



Edge devices



AI book



ARM devices



RISC-V AI chips



Quantum Compute

Quantum Cloud



Edge AI: The ecosystem amplifier for Open Compute

Edge AI brings open models and open compute closer to developers, learners and startups.



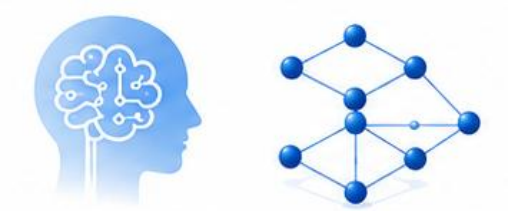


Fast Prototype
Fast Knowledge Spreading
Local Innovation

Edge AI is the practical last mile of Open Compute.

From Open Models to Open Capacity

Why Open Compute exists



AI / Algorithm Innovation

- ✓ Open models accelerate ideas
- ✓ Need open deployment

Open Compute / FlagOS

Open Software for AI Systems

- ↔ Common interfaces
- ≈ Reusable software assets
- ✚ Interoperability



Chip / Compute Innovation

- ✓ Unlocks deployment
- ✓ Reaches devices

✓ Open models without open compute do not create equal access to AI.



OPEN MODEL ACCESS

- ✓ Weights & algorithms available
- ✓ Deployment still depends on compute + software



OPEN CAPACITY

- ✓ Run, adapt and control AI
- ✓ Reuse capability across ecosystems

More AI Tasks Are Becoming Edge-Capable

Extend FlagOS Data Center AI Expertise and Compute Concept to the Edge



Powerful Edge Models

Qwen3.8-27B

SWE-bench Verified	61.3	vs	53.4	Opus 4.6
OSWorld-Verified	84.7	vs	72.7	Opus 4.6

MiniCPM-V 4.5

8.7B params

OpenCompass	77.0	>	GPT-4o-latest

Edge AI Advantages

- **Data Security**
- **Privacy**
- **Low Latency**
- **Low Cost**

Target Platforms for FlagOS Edge

High-Compute SoCs for Embodied AI / AgentBox: Orin / Thor Class

D-Robotics | RIVAI | NIO | Li Auto | Black Sesame | SiEngine | GΞLIX

- Automotive roots; SoC-only designs, with NPUs providing most AI compute.
- Software: automotive production prioritizes low latency and static graphs, with no further development. Final products use highly specialized engines and SDKs.
- New need: a development model for embodied AI and high-compute Agent PCs that balances downstream developers' needs and embraces open ecosystems.

SoCs / Broad-Reach Platforms

CIX | Mac | Qualcomm | SpacemiT | SOPHGO

- Mainstream platforms are SoCs spanning AI PCs and other uses, suited to CPU / iGPU / NPU collaboration.
- Readily available, relatively open and developer-oriented; good for building developer familiarity and serving as DevKit base platforms.
- Software stacks and performance paths are fully integrated with open-source ecosystems.

Full Edge-to-Cloud Product Lines

Iluvatar CoreX | Moore Threads | Tsingmicro | Denglin | AMD | Nvidia

- Cloud roots, mostly GPGPU hardware, with broad software ecosystem support.
- Reuse cloud FlagOS and compiler assets; edge offerings typically already prioritize adapted open frameworks such as llama.cpp and vLLM.
- Development and testing stacks can be decoupled. TensorRT is a reference for final products requiring no further development.

Discrete Edge NPUs

NXP | Rockchip | AXERA | Houmo AI (compute-in-memory)

- Apart from newer startups, most began with low-compute vision models. Products now commonly offer 30–50 TOPS and need LLM / VLM support.
- Mostly closed SDKs, operators and compilers. Interest extends to high-compute AgentBox and even cloud markets; rising compute and diverse needs drive Python DSL interfaces.
- Architectural constraints give NPUs weaker ecosystem compatibility than GPGPUs, requiring more development work.

Use Cases | AgentBox, Industrial Edge & Embodied AI

01

High-Compute AgentBox

Benchmark: DGX Spark
Large models, long context, light fine-tuning and further development
Key needs: open toolchains and model compatibility
Model formats: GGUF and HF

Front End: llama.cpp / Ollama / PyTorch

02

30–100 TOPS Edge

Fragmented industrial, IoT and long-tail customers
Need rapid adaptation and further development

Closed SDKs limit the developer ecosystem and slow adaptation.
Model formats: GGUF and HF

Opportunity: DevKit + Model Distribution

03

Embodied AI

Models and algorithms have yet to converge
4090 development + TensorRT on-device; longer chain for Chinese chips
New operators: CPU fallback or wait for SDK (D-Robotics)

High-performance, low-latency inference: FlashRT-like optimization (RIVAI)

Opportunity: Triton Operators + Native PyTorch Development

FlagOS: Connects AI Silicon to Open Source AI Ecosystem

Northbound framework support brings chip partners into the open-source AI ecosystem

Mainstream model formats

Hugging Face models

Safetensors / PyTorch checkpoints

GGUF

llama.cpp ecosystem

Open-source frameworks

PyTorch
FlagOS integration

vLLM
vllm-plugin-FL

SGL
sglang-plugin-FL

llama.cpp
FlagOS Backend

FlagOS shared compiler and operator backend

FlagGems
Operator library

FlagTree
Unified compiler

KernelGen
AI kernel generation

Multi-vendor hardware

CPU / GPU / NPU

FlagRelease

Model publishing and delivery

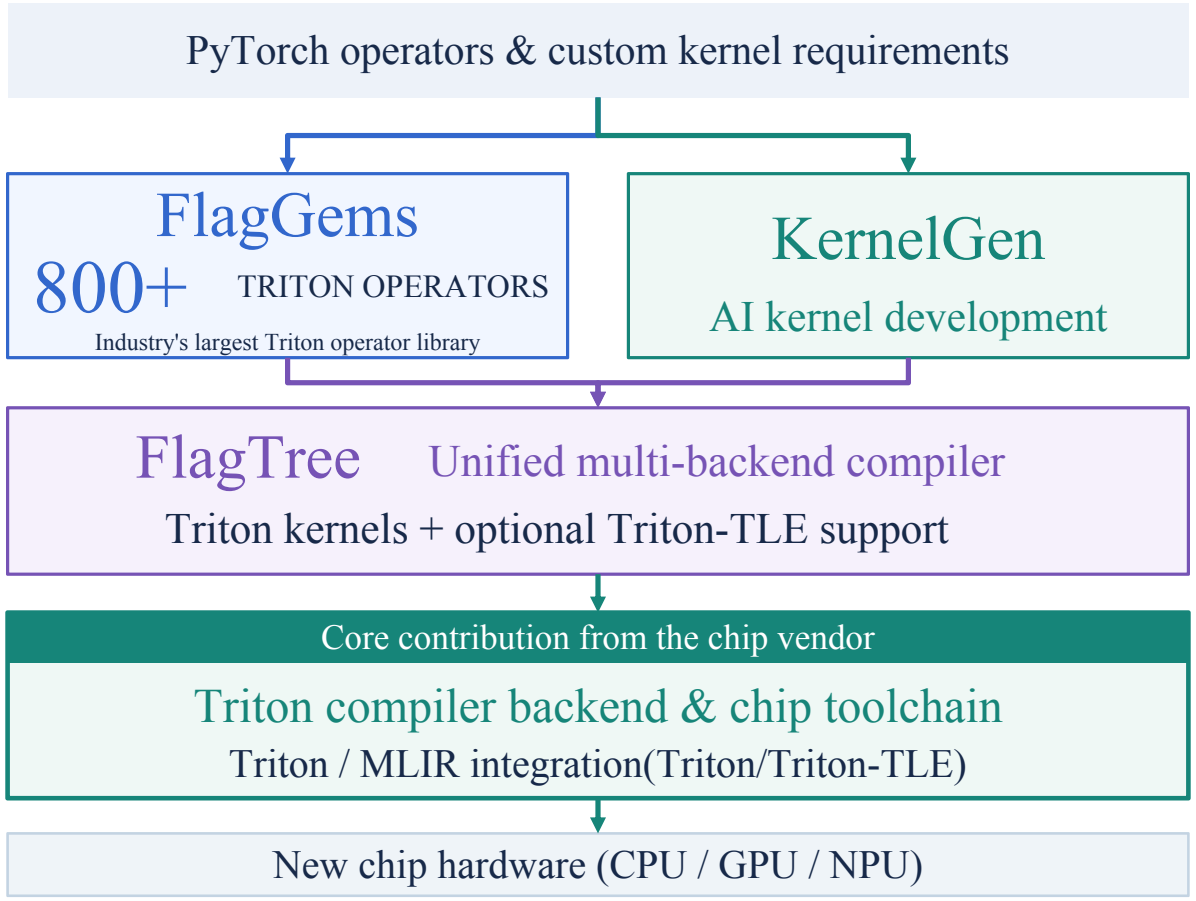
- Model publishing
- Version release
- Product delivery

Help chip partners reach open-source developers and deploy models faster

FlagOS: Unified Compiler and Operator Library

Open operators, kernel generation and compilation for new chip integration

Operator development and compilation



What Chip Vendors Need to **Provide:**

- **Provide Triton compiler support**
  Compiler backend + chip toolchain
- **Enable on-chip execution**
  Drivers, runtime and PyTorch interfaces
- **Integrate the chip backend into FlagTree**
  Buildable packages + version compatibility
- **Maintain and upgrade the Toolchain**

Goal: reuse and generate operators to accelerate model support on new chips

FlagOS software stack Integration and model adaptation timeline from an AI Silicon

01 Core Stack Integration

Jan 2026

Triton 3.4 Integration

FlagTree: compiler support
FlagGems: operator backend support

Apr 2026

Build & Runtime Support

FlagTree: build and runtime support
FlagCX: devices and communication

Apr 30, 2026

DeepSeek-V4 Day0 Support

Model adaptation milestone
Pytroch inferencing

02 Inference & Engineering

May 2026

Inference & Deployment

vllm-plugin-FL: integration
FlagRelease: dedicated installers
FlagGems: 78 standard operators and 6 fused operators

Jun 2026

Coverage & Engineering

FlagCX: PyTorch communication
FlagGems: CI integration, 91 standard operators and 9 fused operators
KernelGen v2.1 supports

Jul 2026

Triton 3.6 Adaptation

FlagTree and FlagGems adaptation
Version stabilization in progress

03 Models & Frameworks

Aug 27, 2026

Qwen3.8-Flash-Next

Model adaptation complete

Aug 28, 2026

glm-5.3-flash

Model adaptation complete

Aug 31, 2026

vllm-plugin-fl 0.20.2

Version integration complete

Sep 9, 2026

MiniCPM5-2B

Day0 Model Adaptation

FlagOS edge ecosystem: current status

FlagOS Open Edge Ecosystem

Compilers / Operators / Inference Engines

Open Ecosystem Direct Support

Apple Mac

AMD

Open Toolchains Enable Broader Platform Support

Completed Integrations

SpacemiT

Operator Library + Compiler Backend Integration

CIX

BOSC (Co-development Partner)

End-to-End Model Inference on FPGA

Full-Stack Integration In Progress

RIVAI

D-Robotics

NIO Shenji Chip

Compiler, Operators and Runtime Joint Integration

Global Vendors Integration Partnerships

Intel

Qualcomm

Renesas

NXP

Ongoing Engagement and Integration

ESWIN Computing

China Mobile XinSheng

Tongxin Microelectronics

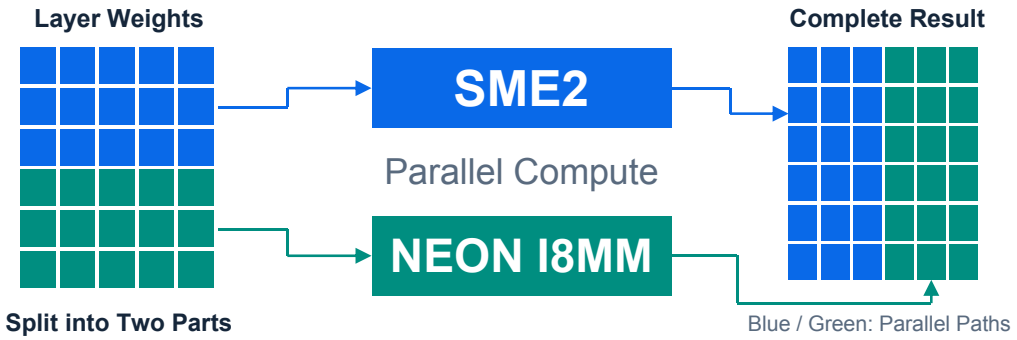
SigmaStar

Together supporting edge use cases: AI PC, AI Box, embodied AI and in-vehicle systems

FlagOS Optimization on Arm CPU Platform

SME2 + I8MM: Joint GEMM Compute Accelerates Prefill

Partition one layer's weights; compute with SME2 and I8MM concurrently



+21%–25% **Full-Model Prefill Throughput Gain**

PP512 +21.3% · PP1024 +24.3% · PP2048 +24.5%

MiniCPM5-2B W8A8 · M5 Pro · batch=1 ·

Key Operator Optimizations (vs. Aten)

Operator	vs. Aten Operator
RMSNorm	2.12–2.78×
SwiGLU	1.30–1.31×
RoPE	8.71–8.86×
Activation Quantization & Packing	4.22–4.38×
GEMV (W8A8)	3.14–4.08×
Softmax	1.14–1.16×

Arm CPU: Supported Models and Performance

Tested on Mac M5 Pro, CPU-only

Model	Quant.	Prefill (pp512, tok/s)	Decode (tg128, tok/s)	Total (tok/s)	vs llama.cpp (Total)	vs vLLM Native (Total)
MiniCPM5						
MiniCPM5-1B	W8A8	2809.72	123.49	528.3	-3.13%	+96.7%
MiniCPM5-2B	W4A8	1017.78	91.66	332.35	+9.17%	+39.09%
MiniCPM5-2B	W8A8	1146.82	64.91	265.65	+13.77%	+174.2%
Qwen3.8-27B						
FlagOS no-spec	W4A8	74.93	12.73	38.07	+28.22%	+83.9%
FlagOS DFlash2	W4A8	74.5	23.4	52.03	—	—
MoE Model						
Xing4.0 29B-A4B	W4A8	312.48	38.97	118.97	—	+252.09%
Ling-3.0-tiny 7.9B-A1.3B	W8A8	1,209.38	83.83	334.60	+2.25%	—

FlagTree-CPU 独立发版，编译、低比特算子与运行时协同增强。

Compile

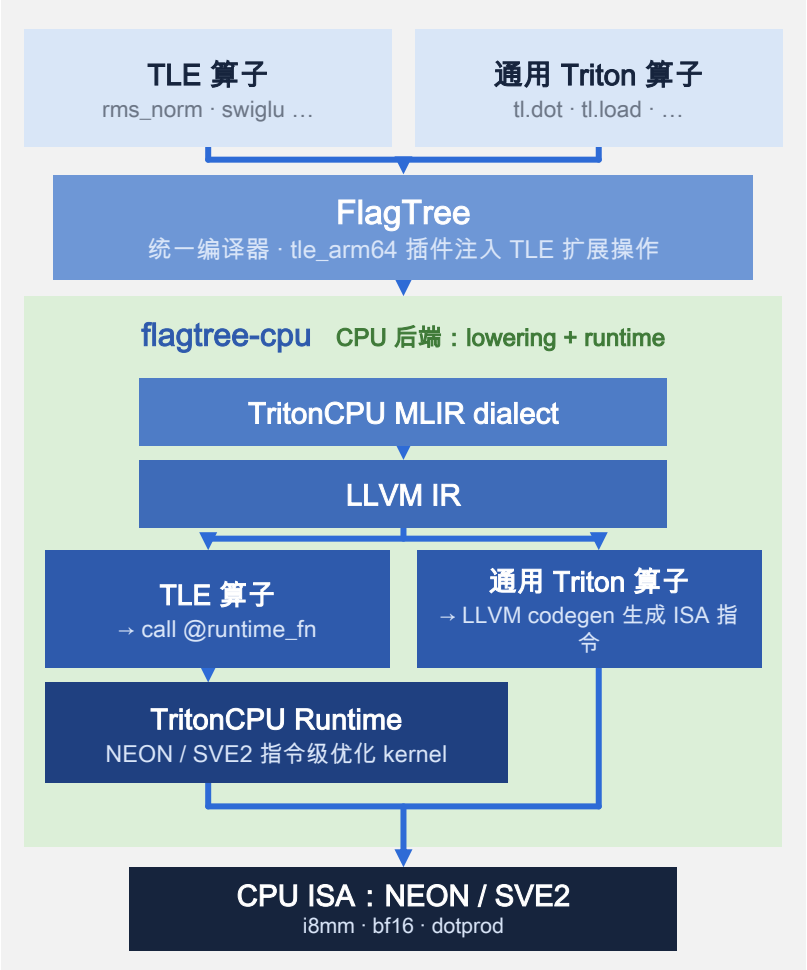
SVE2 / NEON I8MM
W4A8 / W8A8 低比特路径

Operator

权重预打包与融合
匹配 Prefill / Decode 计算形态

Runtime

线程调度与编译内核复用
减少解码热路径开销



Qwen3.8-27B
W4A8 · 纯 CPU

74.93

Prefill · tok/s

12.73

Decode · tok/s

Apple M5 Pro · 64 GiB
Batch=1

FlagOS Edge: Developer-Friendly Platform for Product Innovation

FlagOS

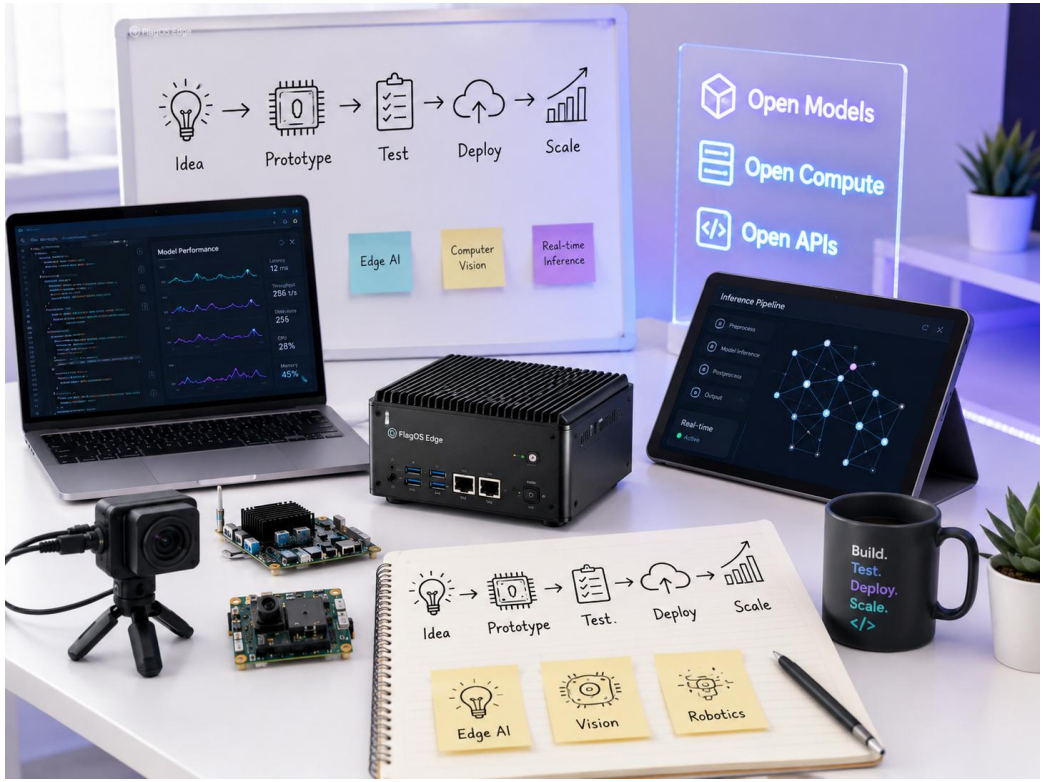
Open models, unified APIs and reusable software assets — everything developers need.

Build for Developers

- Open Models
- OpenAI-Compatible API
- SDKs & Inference APIs
- One-click Deployment

FlagOS Edge: North Bound Ready AI Solution

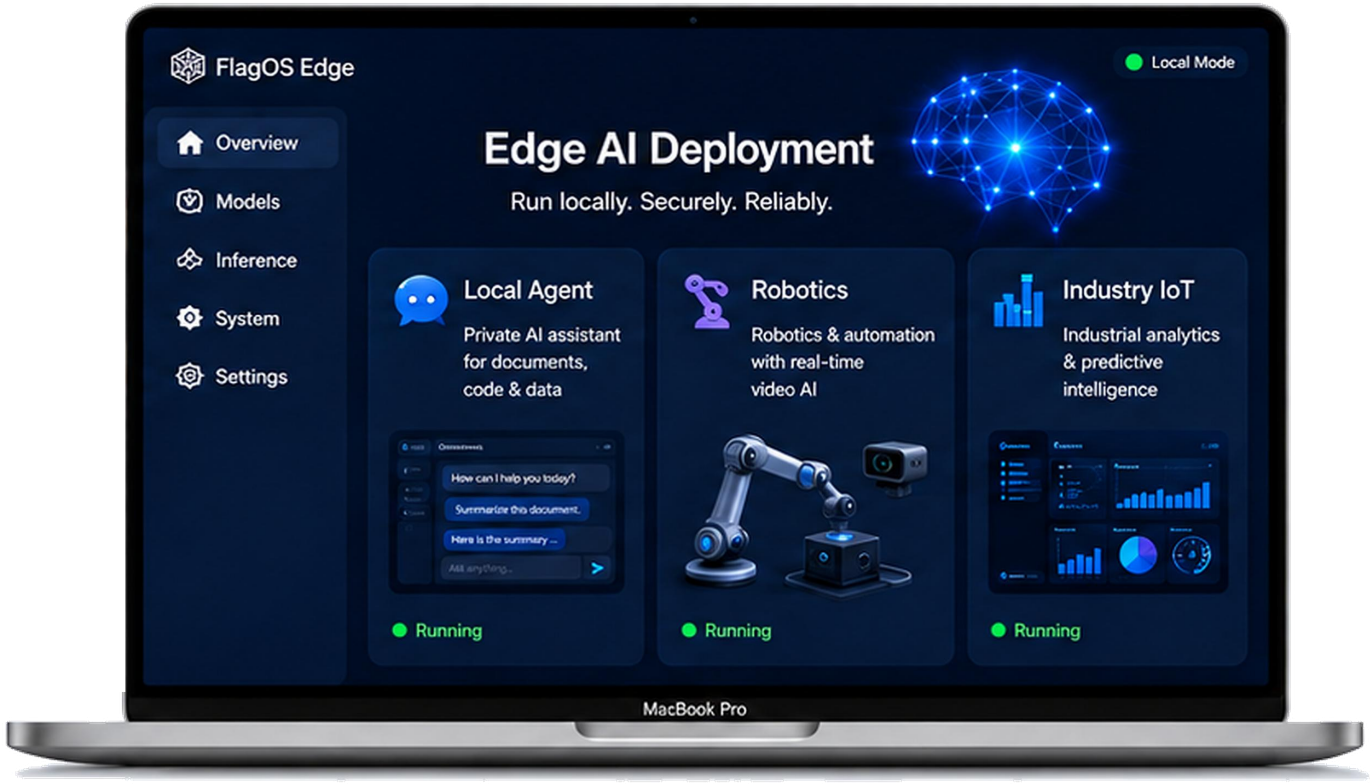
- Production Ready
- Performance Ready
- Variable Hardwares



Open-Box Experience · Fast Deployment · Reliable Maintenance

FlagOS in Edge: Broad Models, Proven Performance

High-performance, out-of-the-box inference for local AI products and application innovators



FlagOS Edge: Open-box Performance Solution

Qwen3.8 27B W4A8

Mac M5 Pro · 18-core CPU · Batch=1 · CPU-only

74.93 Prefill tok/s	**12.73** Decode tok/s
38.07 E2E tok/s	**27.65** Decode MTP tok/s

Representative Open Models

Qwen · MiniCPM · Gemma · Glimmer · Embodied Models

FlagOS Edge: From Chip Enablement to Product Enablement

A unified AI software stack bridging open models, inference frameworks and heterogeneous edge hardware.

FlagOS

APPLICATIONS & AGENTS

Agentic Apps

Embodied / Industry

Local AI Services

OPEN MODELS

Hugging Face Models

GGUF Models

FlagOS

Unified AI Software Stack for the Edge

FlagRelease

Model Publishing

Version Release

Product Delivery

Inference Ecosystem

vLLM-plugin-FL

SGLang-plugin-FL

llama.cpp FlagOS Backend

FlagTree Unified Compiler

FlagGems Unified Operator Library

Hardware Abstraction & Optimization

HETEROGENEOUS COMPUTE

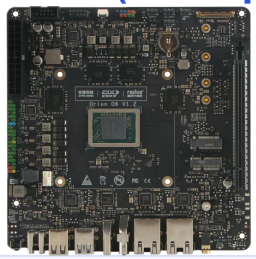
CPU

GPU

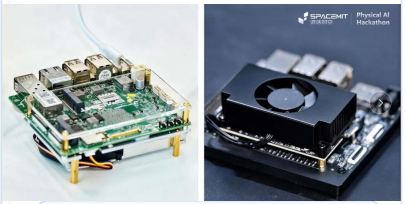
NPU

One best-practice Edge stack — across models, frameworks and hardware.

Supported Devices(Example)



CIX P1



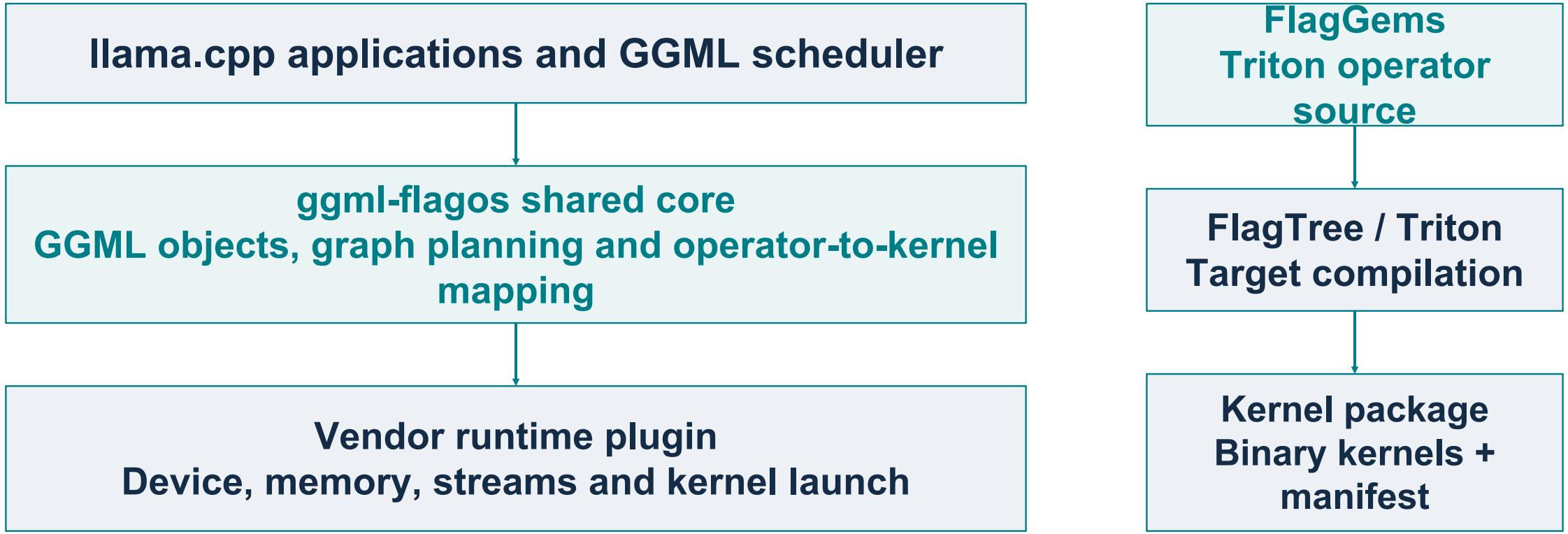
SpacemiT K3



More AI PC/Box

ggml-flagos: a shared backend core

A common core separates GGML integration from vendor runtime and kernel delivery.



Core and plugins use a C interface. Kernel packages ship separately and carry target-specific variants.

Source: ggml-flagos vendor deck, slide 1 and presenter notes.

Vendor integration and runtime reuse

Two deliverables: a runtime plugin and a target kernel package

Runtime plugin

17 required C functions
Device, memory, copy, stream and kernel services
Optional capture, zero-copy and fusion
Binary-only plugin delivery is supported

Kernel package and acceptance

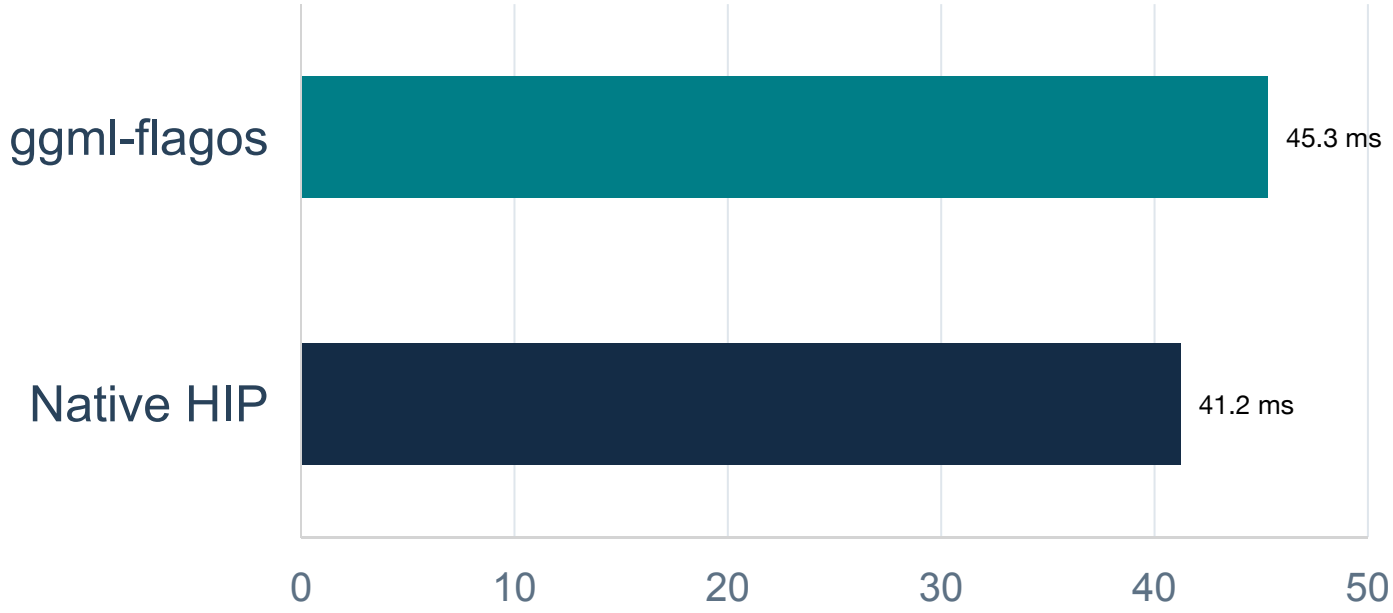
Compiled kernels with a manifest
Operator tests must pass
Full-model divergence target: KLD < 0.01
Record a reproducible llama-bench baseline

Runtime phase	When it runs
Plugin discovery and model loading	Once at startup / model load
Placement and execution planning	When graph structure changes
Argument binding and kernel launch	For each token; graph replay when supported

Source: ggml-flagos vendor deck, slides 2–3 and presenter notes.

ggml-flagos: measured decode gap and next work

AMD 890M, Qwen3.5-4B Q4_K_M. Reported milliseconds per token, lower is better.



Reported breakdown

44.7 ms GPU kernel execution
0.6 ms GPU idle time

Next optimization work

Tune GDN and SSM_CONV
Rerun full-model measurements
Validate more backends

The 98–99% native-speed figure in the source is a projection. No post-optimization result is claimed here.

Source: ggml-flagos vendor deck, slide 4. Rounded timings are source-reported project measurements.

Two partner platforms test different integration paths

CIX P1 and SpaceMIT K3 make the FlagOS reuse problem concrete on Arm and RISC-V.

CIX P1 · Arm CPU path

An Arm64 application ecosystem

Quantized CPU operators and multicore tuning

FlagGems, FlagTree-CPU and the vLLM plugin

SpaceMIT K3 · RISC-V path

Vector and tensor compute through the ISA

Compiler lowering and runtime integration

Triton, spine and common operator interfaces

Shared interfaces can carry over; ISA lowering, kernels and runtime validation remain target-specific

CIX P1: a verified Arm CPU integration

CIX reports joint P1 adaptation and optimization for FlagOS 2.1's native Arm64 CPU support.

Platform facts



12-core Armv9.2
8 × A720 + 4 × A520

Up to 64 GB memory

10-core Immortalis-G720 GPU
30 TOPS NPU; 45 TOPS combined*

Documented FlagOS integration

FlagGems quantized operators
FlagTree-CPU kernel compilation
vLLM plugin CPU platform

Published CIX test results

9/9 FlagGems tests
16/16 vLLM plugin tests

The evidence covers CPU execution and integration. NPU and full-model speed claims need separate tests.

*Vendor peak specifications, not a measured application rate. Photo: CIX / FlagOS, 4 Aug 2026. Test scopes and sources in notes.

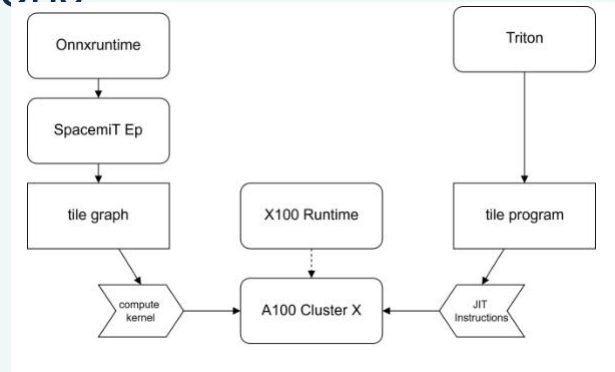
SpaceMIT K3: a RISC-V compiler and operator path

Architecture preview, 29 Jan 2026. Public source integrations establish a separate software evidence trail.

K3 architecture preview

8 X100 cores + 8 A100 AI-oriented cores

One Linux system; vector and tensor extensions



Verified integration and open work

FlagTree #933: merged Aug 2026

K3 RVV backend using spine-mlir and spine-runtime, with CI

FlagGems #3793: merged Jun 2026

Triton update; mm/bmm, GELU and argmax integration work

ggml-flagos status: build passed; runtime plugin in progress. No K3 end-to-end speed result is reported here.

Sources: SpaceMIT K3 preview p.14; FlagTree and FlagGems pull requests; supplied FlagOS project status. Links and scope in notes.

A reproducible pilot across CIX and SpaceMIT

The teaching experiment keeps the workload stable and changes one execution path at a time.

COMMON BASELINE

Fixed model revision

Fixed input and precision

Working CPU baseline

Pinned software versions

PLATFORM INTEGRATION

CIX: documented Arm CPU path

SpaceMIT: named K3 backend

Operator and model tests

COMPARABLE EVIDENCE

Quality and failures

Prefill and decode

Peak memory and power

Reproducible test logs

Next milestone: a reproducible report for each target before broader deployment claims

Discussion and takeaways

Placement Local, nearby edge and cloud paths can work together.

Execution Model support, memory and data movement shape performance.

Systems Applications need validation beyond a single model benchmark.

What would you test first in your own teaching or research?

Appendix: a shared vocabulary

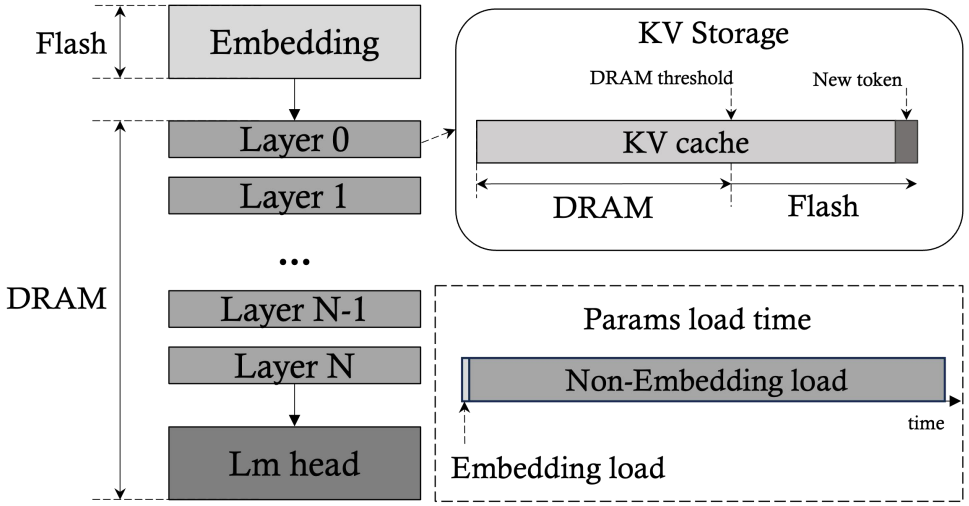
ISA	The instruction set that software targets, such as Arm or RISC-V
SoC	A chip that combines processors, interfaces and other functions
NPU	A processor or accelerator specialized for neural workloads
Runtime / engine	Software that loads or compiles a model and manages execution
Operator / kernel	A model operation and its concrete implementation
Prefill / decode	Processing the input context / generating subsequent tokens
KV cache	Stored attention state used during language-model generation

MNN: mobile runtime and Arm kernel integration

Alibaba MNN / MNN-LLM pairs model execution with memory and kernel optimizations.

A public implementation at MWC 2025

Qwen2-VL-2B-Instruct on vivo X200 Series
MediaTek Dimensity 9400, using Arm CPU + KleidiAI



MNN-LLM research: weights and KV cache across DRAM / Flash

Arm + Alibaba

Joint KleidiAI integration
Optimized matrix multiplication for the demonstrated CPU path

MediaTek

Dimensity 9400 supplies the SoC
The cited demo uses its Arm CPU
NPU execution is a separate path

Other deployment paths

MNN Chat: Android example app
Optional QNN backend support
Check model support

Sources: Arm + Alibaba MWC demo (Mar 2025), MNN Android app and NPU docs. Original MNN-LLM Fig. 1 (arXiv 2025).

Original vendor materials

Public slide decks and technical references used in this presentation.

Arm

Vision Transformers at the Edge

2024 / PDF pp.8–14

Arm + Meta

ExecuTorch and KleidiAI

2025 / Technical article

NXP

i.MX and discrete AI accelerators

2025 / PDF pp.3, 14–16

STMicroelectronics

ST Edge AI Suite overview

PDF pp.2, 7–8

Andes

AI on RISC-V

2025 / PDF pp.10–21

SiFive

Leadership in the New Computing Era

2025 / PDF pp.23–27

SpacemiT

K3 architecture preview

2026 / PDF pp.3, 14

Arm at Linaro Connect

Making KleidiAI work for your framework

2025 / Session page

Page numbers refer to the PDF file order, starting at page 1.